22360859088
KORAY GARİP

BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Node.JS ile Web Programlama

LAB_7

22360859088
KORAY GARİP

1. Callback Fonksiyonu Mantığı

Bu aşamada callback fonksiyonu kavramı incelendi. Callback, bir fonksiyonun parametre olarak başka bir fonksiyonu alması ve işlem tamamlandıktan sonra bu fonksiyonu çalıştırmasıdır. Özellikle asenkron işlemlerde sonuç döndürmek yerine callback kullanıldığı gözlemlendi.İlk olarak basit örnekler ile callback mantığı pekiştirildi. setTimeout kullanılarak gecikmeli çalışan fonksiyonlar yazıldı ve işlem tamamlandıktan sonra callback fonksiyonunun çalıştığı görüldü. Bu sayede asenkron işlemlerde verinin doğrudan return edilemediği, bunun yerine callback ile geri döndürüldüğü anlaşılmıştır.

Örnek:

```
const geo=(address,callback)=>{
    setTimeout( ()=>{
    const data={
        lat:123,
        lon:55
    }
    callback(data)
    },2000)
};
geo("bursa",(data)=>{
    console.log(data)
});


const add=(s1,s2,callback)=>{
    setTimeout( ()=>{
        const sum=s1+s2
        callback(sum)
    },2000
)
}
add(1,4,(sum)=>{
    console.log(sum);
})
```

2. Geocoding ve Forecast Fonksiyonlarının Callback ile Yazılması

İki temel fonksiyon oluşturuldu:

- geo → Girilen adresi koordinata (lat, lon) çevirir
- forecast → Koordinata göre hava durumu bilgisini getirir

Her iki fonksiyon da asenkron çalıştığı için sonuçları callback ile döndürmektedir.

Geo Fonksiyonu:

- API üzerinden konum bilgisi alınır
- Hata kontrolü yapılır
- Başarılı durumda {lat, lon, location} bilgisi callback ile döndürülür

Forecast Fonksiyonu:

- Weather API'ye istek atılır
- Gelen veriler işlenir
- Sıcaklık, hissedilen sıcaklık ve yağış durumu yorumlanarak mesaj oluşturulur
- Sonuç callback ile geri döndürülür

22360859088
KORAY GARİP

Örnek: utils/geocode.js

```javascript
const request = require("postman-request");


const geo = (address, callback) => {
    const geourl2 = `https://geocoding-api.open-meteo.com/v1/search?name=${address}&count=10&language=en&format=json`;

    request({ url: geourl2, json: true }, (err, res, body) => {

        if (err) {
            callback("İnternet Hatası: " + err.code,undefined);
            return;
        }
        if (body.results.length === 0) {
            callback("url bilgilerini kontrol etmelisin veya sonuç bulunamadı", undefined);
            return;
        }

        const lat = body.results[0].latitude;
        const lon = body.results[0].longitude;
        const location=body.results[0].name
        callback(undefined, { lat, lon, location });
    });
};

module.exports = geo;

```

Örnek: utils/forecast.js

```javascript
const request=require("postman-request");

const forecast = (lat, lon, callback) => {
    const url = `http://api.weatherstack.com/current?access_key=2f555b90f43de6969ac39210ee6559a9&query=${lat},${lon}`;

    request({ url, json: true }, (err, res, body) => {
        if (err) {
            callback("Hava durumu servisine bağlanılamadı!", undefined);
        } else if (body.error) {
            callback("Konum bulunamadı, bilgileri kontrol edin.", undefined);
        } else {
            const temp = body.current.temperature;
            const feelTemp = body.current.feelslike;
            const precip = body.current.precip;
            const weather = body.current.weather_descriptions[0];

            let message = `${weather}. Hava ${temp} derece ve hissedilen sicaklik ${feelTemp} derece. `;
            if (precip === 0) message += "Yağış beklenmiyor.";
            else if (precip <= 10) message += "Az yağış bekleniyor.";
            else if (precip < 25) message += "Ortalama yağış bekleniyor.";
            else if (precip < 50) message += "Sağanak yağış bekleniyor.";
            else message += "Fırtına bekleniyor.";

            callback(undefined, message);
        }
    });
};

module.exports = forecast;
```

3. Callback Zinciri

Ana dosyada (app.js) bu iki fonksiyon birbirine bağlı şekilde çalıştırıldı.

Çalışma sırası:

1. Kullanıcıdan komut satırından adres alınır
2. geo fonksiyonu çağrılır
3. Koordinatlar alındıktan sonra forecast fonksiyonu çalıştırılır
4. Sonuç kullanıcıya yazdırılır

Bu yapı sayesinde bir asenkron işlemin sonucu diğerine aktarılmış olur.

Örnek: app.js

```
const geo = require('./utils/geocode');
const forecast = require('./utils/forecast');

const address = process.argv[2];

if (!address) {
    console.log("Bir konum girin. Örnek kullanim node app.js Bursa");
} else {

    geo(address,(gerr,gdata)=>{
    if(gerr) return console.log("Hata",gerr)

    forecast(gdata.lat,gdata.lon,(ferr,fdata)=>{
        if(ferr) return console.log("hata",ferr)

        console.log(fdata)

        })

    })
}
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
PS C:\Users\koray\Desktop\nodeproject\weather> node .\app.js Mudanya
Sunny. Hava 15 derece ve hissedilen sicaklik 15 derece. Yağış beklenmiyor.
PS C:\Users\koray\Desktop\nodeproject\weather>
```

callback(error, data)

- Eğer hata varsa → error dolu gelir
- Eğer işlem başarılıysa → data dolu gelir

Bu sayede her aşamada hata kontrolü yapılabilmiştir. Örneğin:

- internet hatası
- API hatası
- veri bulunamama durumu

gibi durumlar kullanıcıya uygun mesajlarla iletilmiştir.

4.Object Destructuring

Bu aşamada JavaScript'te object destructuring (nesne parçalama) konusu incelenmiştir. Bu yapı sayesinde bir nesne içerisindeki verilere tek tek erişmek yerine, daha kısa ve okunabilir şekilde değişkenlere atama yapılabilmektedir. Özellikle fonksiyon parametrelerinde kullanılarak kodun sadeleşmesi sağlanmaktadır.

Örnek:

```
const product={
    name:"Laptop",
    price:12345,
    stock:678,
    salePrice:undefined
}

const transcation=(type,product)=>{
    const {name,price,stock} = product;
    console.log(type,name,price,stock);
}
transcation("Sipariş",product)
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
PS C:\Users\koray\Desktop\nodeproject> node .\lab.js
Sipariş Laptop 12345 678
PS C:\Users\koray\Desktop\nodeproject>
```